

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. finRGB. "Standard Brownian Motion / Wiener Process: An Introduction." *YouTube*, 13 July 2019, <https://www.youtube.com/watch?v=ld0rxwAJpkM>.

- **Required pages:** Entire video
- **Estimated reading time:** 20 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 1

THE TRINOMIAL MODEL

Derivative Pricing - Module 3



Outline

- ▶ Extending the Binomial Model
- ▶ Revisiting No-Arbitrage Argument in Trinomial Trees
- ▶ Pricing Problems Associated with Trinomial Model

Extending the Binomial Model

In the previous modules, we dealt extensively with the different features associated with pricing various types of options in the binomial model.

In this module, we extend the binomial model to consider different types of stock price movements:

- ▶ Ideally, you would like a continuous-time framework in which underlying stock prices can move any direction in any magnitude to capture the underlying behavior.
- ▶ While we will get there through the course, there are many problems that arise once we depart from the binomial model setting.
- ▶ In order to **better understand what these are and why they arise**, we will naturally extend the binomial model to a **trinomial model**:

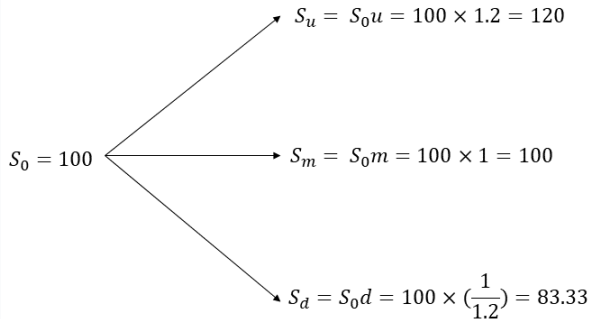
⇒ That is, prices could move in 3 different directions:

- ▶ Upward movement, u .
- ▶ Mid (or no-) movement, m . (typically $m = 1$)
- ▶ Downward movement, d (typically $d = \frac{1}{u}$).

The Trinomial Model

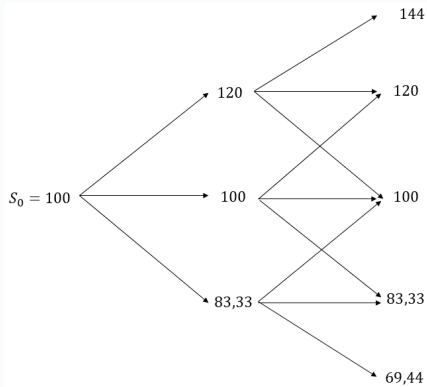
Here, you have an example of how a trinomial model with 1 step ($N = 1$) will look:

$u = 1.2$; $d = \frac{1}{u}$; $m = 1$; $N = 1$; and $S_0 = 100$



The Trinomial Model

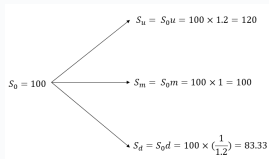
Here, you have an example of the same trinomial tree with $N=2$:



No-Arbitrage in Trinomial Model

Let's now dig deeper on how to apply risk-neutral valuation in the Trinomial tree.

We will deal with 3 probabilities here: p_u , p_m , and p_d , associated with corresponding movements:



From absence of arbitrage and risk-neutral probabilities, these are the conditions that have to be met:

$$(1) \quad p_u + p_m + p_d = 1$$

$$(2) \quad S_0 = e^{-rdt} [p_u \times S_u + p_m \times S_m + p_d \times S_d], \rightarrow \text{Martingale condition (more on this later)}$$

$\Rightarrow$ Does this system of equations have a solution?

No-Arbitrage in Trinomial Model: FTAP I

The previous system of equations has, indeed, multiple solutions.

- Because $\# \text{ unknowns} > \# \text{ equations}$.
- There exist multiple sets $\{p_u, p_m, p_d\}$ that constitute a solution.
- For any of these sets of $\{p_u, p_m, p_d\}$, we can say such a market is arbitrage-free.

What we have implicitly derived here is the first Fundamental Theorem of Asset Pricing (**FTAP I**)

Fundamental Theorem of Asset Pricing (FTAP) I

If there exists a set of measures $\{p_u, p_m, p_d\}$, the market is arbitrage-free.

But for our pricing purpose, another important problem arises:

⇒ How can we select the set of risk-neutral probabilities to use among infinite measures?

Summary of Lesson 1

In lesson 1, we have:

- ▶ Extended the binomial tree to augment diversity of stock price movements.
- ▶ Introduced the trinomial tree as a natural next step.
- ▶ Gotten a first glance at the potential problems with replication in a trinomial setting.

⇒ **TO-DO NEXT:** Now, please go to the associated **Jupyter Notebook** for this lesson to check that you know how to build a trinomial tree using Python. This is not as easy as it is for the binomial model. You will also see there an example on how to match volatility to stock price movements in a trinomial tree.

⇒ In the next lesson, we will continue to deal with the problems associated with replication in trinomial trees.

Lesson 1 - The Trinomial Model

February 8, 2026

0.1 DERIVATIVE PRICING

MODULE 3 | LESSON 1

1 THE TRINOMIAL MODEL

Reading Time	60 minutes
Prior Knowledge	Binomial model, Option pricing, No-arbitrage
Keywords	Trinomial model, Matching volatility

*Now that we are comfortable with the dynamics of the binomial model, we are going to extend this pricing framework to a more complex level by considering a **trinomial model**. In this lesson, we are going to introduce the trinomial model in Python and learn how to develop it adapting the same features from the binomial case. Even though everything is pretty similar, please make sure you understand the code fully, as this will be important later on.*

1.1 1. Trinomial Tree: Underlying Evolution (Toy-Example)

Intuition: How do we get the underlying evolution in a trinomial recombining tree?

Note that the tree is **recombining**, which means $d = \frac{1}{u}$. This is an assumption that is not purely necessary but simplifies our pricing task, as we will see when we discuss the completeness of the model.

As always, let's start importing the necessary libraries:

```
[1]: import numpy as np
```

Consider a trinomial tree similar to the one from the slides with just one period ($T = 1$) and one step in the pricing process ($N = 1$). Also, with the following parameters: $u = 1.2$, $d = 1/u$, $m = 1$ and $S_0 = 100$. All rights reserved WQU WorldQuant University QQQQ

The following code snippet computes the underlying values at the end of the “nb-period,” in this case, as we are dealing with a 1-period tree, at the end of the first period:

```
[2]: nb = 3 # number of periods (T=1)
up = 1.2 # up movement
down = 1 / up # down movement to force a "recombining tree"
s0 = 100

vec_u = up * np.ones(nb)
np.cumprod(vec_u, out=vec_u) # Computing u, u^2, u^3....u^nb

vec_d = down * np.ones(nb)
np.cumprod(vec_d, out=vec_d) # Computing d, d^2, d^3....d^nb

res = np.concatenate(
    (vec_d[::-1], [1.0], vec_u)
) # putting together the last period tree underlyings ("nb-period")
res *= s0 # multiplying all by S0
print(res)
```

```
[ 57.87037037  69.44444444  83.33333333 100.          120.
 144.          172.8         ]
```

In the case of having a two-step model, this would be:

```
[3]: nb = 2 # number of periods (T=2)
up = 1.2 # up movement
down = 1 / up # down movement to force a "recombining tree"
s0 = 100

vec_u = up * np.ones(nb)
np.cumprod(vec_u, out=vec_u) # Computing u, u^2, u^3....u^nb

vec_d = down * np.ones(nb)
np.cumprod(vec_d, out=vec_d) # Computing d, d^2, d^3....d^nb

res = np.concatenate(
    (vec_d[::-1], [1.0], vec_u)
) # putting together the last period tree underlyings ("nb-period")
res *= s0
print(res)
```

```
[ 69.44444444  83.33333333 100.          120.          144.         ]
```

At this point, the only thing we have printed is the terminal values of the underlying stock price.

Finally, if we want to get the evolution of the underlying stock for each step in the tree, we can use the following functions together:

(Note: It is important to understand these functions in order to succeed in the next lesson).

```
[4]: def _gen_stock_vec(nb, up):
    s0 = 100
    down = 1 / up
    vec_u = up * np.ones(nb)
    np.cumprod(vec_u, out=vec_u) # Computing u, u^2, u^3...u^nb

    vec_d = down * np.ones(nb)
    np.cumprod(vec_d, out=vec_d) # Computing d, d^2, d^3...d^nb

    res = np.concatenate(
        (vec_d[::-1], [1.0], vec_u)
    ) # putting together the last period tree underlyings
    res *= s0
    return res
```

```
[5]: nb = (
    5 # In this example, np means "periods" because we are not matching
    ↪ volatility yet.
)

for i in range(1, nb + 1):
    vec_stock = _gen_stock_vec(nb - i, 1.2)
    print(vec_stock)
```

```
[ 48.22530864  57.87037037  69.44444444  83.33333333 100.
 120.          144.          172.8          207.36         ]
[ 57.87037037  69.44444444  83.33333333 100.          120.
 144.          172.8         ]
[ 69.44444444  83.33333333 100.          120.          144.         ]
[ 83.33333333 100.          120.         ]
[100.]
```

1.2 2. Matching Volatility in the Trinomial Tree

In the next step, we are going to revisit an old idea from the previous modules: **matching underlying volatility to the parameters in the tree**.

At this point, you already know about the intuition behind this idea. The situation is a little bit more complex in the trinomial tree than for the binomial case:

```
[6]: s0 = 100
sigma = 0.3
T = 1
nb = 10
h = (
    T / nb
```

```

) # We now consider the number of time-step in each period (this would be our
  ↪ 'dt' )
up = np.exp(sigma * np.sqrt(2 * h)) # We match the volatility
down = 1 / up # Down movement to force a "recombining tree"

vec_u = up * np.ones(nb)
np.cumprod(vec_u, out=vec_u) # Computing u, u^2, u^3....u^nb

vec_d = down * np.ones(nb)
np.cumprod(vec_d, out=vec_d) # Computing d, d^2, d^3....d^nb

res = np.concatenate(
    (vec_d[::-1], [1.0], vec_u)
) # Putting together the last period tree underlying evolution
res *= s0
print(res)

```

```

[ 26.1416388  29.89506684  34.18741373  39.09605769  44.7094869
  51.12889477  58.47000405  66.86515303  76.46568122  87.44465748
 100.          114.35804414 130.77762259 149.55473137 171.02786571
195.58412216 223.66617674 255.78026512 292.50530849 334.50334979
382.5314884 ]

```

In order to easily use this later on, let's put all this into a function, `_gen_stock_vec()`:

```

[7]: def _gen_stock_vec(nb, h):
    s0 = 100
    sigma = 0.3
    up = np.exp(sigma * np.sqrt(2 * h))
    down = 1 / up # down movement to force a "recombining tree"

    vec_u = up * np.ones(nb)
    np.cumprod(vec_u, out=vec_u) # Computing u, u^2, u^3....u^nb

    vec_d = down * np.ones(nb)
    np.cumprod(vec_d, out=vec_d) # Computing d, d^2, d^3....d^nb

    res = np.concatenate(
        (vec_d[::-1], [1.0], vec_u)
    ) # putting together the last period tree underlyings
    res *= s0
    return res

```

And check how it performs with an example:

```

[8]: nb = 3 # 3-steps in 1 "year" (assuming parameters are all annualized)
    h = 1 / nb
    for i in range(1, nb + 1):

```

```
vec_stock = _gen_stock_vec(nb - i, h)
print(vec_stock)
```

```
[ 61.26889168  78.27444773 100.          127.75561233 163.21496482]
[ 78.27444773 100.          127.75561233]
[100.]
```

1.3 3. Conclusion

You now have the basic understanding of how a trinomial tree works. Now, try and use the previously coded function to price options with longer time-steps (or periods)... are you able to? Try to think of a few examples and implement them.

In the next lesson, we will go over pricing on the trinomial tree and the martingale concept to obtain risk-neutral probabilities in the trinomial setting.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.6
- **Estimated reading time:** 30 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 2

PRICING IN THE TRINOMIAL MODEL

Derivative Pricing - Module 3



Outline

- ▶ Overview of Option Pricing in a Trinomial Model
- ▶ The Martingale Concept.
- ▶ Using Risk-Neutral Probabilities in a Trinomial Setting to Force Completeness!

Option Pricing in a Trinomial Model

We have already seen the problems related to extracting risk-neutral probabilities in order to perform option pricing in a Trinomial tree.

Apart from these, pricing is analogous to the Binomial model case! → **Backwards induction**

Let's start with an overview of the pricing problem, which is extracting risk-neutral probabilities.

We know the **2 conditions** that must meet in order to get an arbitrage-free market according to FTAP I:

$$(1) \quad p_u + p_m + p_d = 1$$

$$(2) \quad S_0 = e^{-rdt}[p_u \times S_u + p_m \times S_m + p_d \times S_d]$$

Condition (2) is generally called the **Martingale condition** → What is a Martingale?

- A **Martingale** is a stochastic process for which the conditional expectation of its future value, given the information known up today, is the current value today. Formally:

$$E(X_{t+1}|F_t) = X_t, \text{ where } F_t = \{X_0, X_1, \dots, X_t\}$$

The concept of Martingale

A **Martingale** is a stochastic process such that:

$$E(X_{t+1}|F_t) = X_t, \text{ where } F_t = \{X_0, X_1, \dots, X_t\}$$

- One implication of this is the following:

$$E(X_{t+1} - X_t|F_t) = E(X_{t+1}|F_t) - X_t = 0 \rightarrow \text{Expected Return} = 0$$

- **Example:** Think of a 'fair casino' (unlike in real life). You bet on a color (red/black) and there is a 50% probability of each outcome. You bet \$1 and win another \$1 if you guessed right and lose the bet if wrong.

→ What is the expected return of such a bet?

- In our setting:

With risk-neutral probabilities, the only return made by an investor must be the risk-free rate,

Obtaining Risk-Neutral Probabilities: Complete Market?

We already know the requirements for absence of arbitrage in a market, the first step towards our pricing.

- ▶ But with only the previous 2 equations, we do not have a unique set of probabilities, $\{p_u, p_d, p_m\}$.
- ▶ A set of risk-neutral probs $\{p_u, p_d, p_m\}$ is formally referred to as **Equivalent Martingale Measure (EMM)**.
- ▶ When we do **not have a unique EMM**, we say that the **market is not complete**.
 - Because there is not perfect (unique) replicability of payoffs given multiple EMMs.

Fundamental Theorem of Asset Pricing (FTAP) II

Market completeness (perfect replicability) is only achieved when there is a unique EMM.

⇒ Thus, we need to **"force" completeness** somehow in order to have a unique EMM for our pricing

→ In short, we need another equation for the previous system

Risk-Neutral Probabilities: Forcing Completeness

There are different ways to achieve completeness in a Trinomial tree (e.g., adding an extra risky asset)

Here, we will implement the most commonly used in practice: matching the second moment of the distribution.

- ▶ This is, essentially, a matching volatility approach $\rightarrow (3) \ E(\text{Var}(S_{t+1}|S_t)) = \text{Var}(S_t)$
- ▶ This and forcing a recombining tree ($d = \frac{1}{u}$) will produce the following values:

$$u = e^{\sigma\sqrt{2dt}}, \ d = e^{-\sigma\sqrt{2dt}}, \ m = 1$$

- ▶ Which, in turn, will produce the following expressions for risk-neutral probabilities:

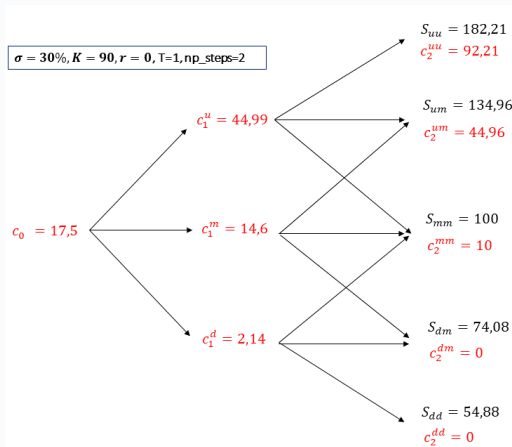
$$p_u = \left(\frac{e^{\frac{rdt}{2}} - e^{-\sigma\sqrt{\frac{dt}{2}}}}{e^{\sigma\sqrt{\frac{dt}{2}}} - e^{-\sigma\sqrt{\frac{dt}{2}}}} \right)^2$$

$$p_d = \left(\frac{e^{\sigma\sqrt{\frac{dt}{2}}} - e^{\frac{rdt}{2}}}{e^{\sigma\sqrt{\frac{dt}{2}}} - e^{-\sigma\sqrt{\frac{dt}{2}}}} \right)^2$$

$$p_m = 1 - p_u - p_d$$

Pricing in the Trinomial Tree

Here, you have a pricing example of a call option with underlying asset $S_0 = 100$ in a trinomial tree:



Summary of Lesson 2

In lesson 2, we have:

- ▶ Understood pricing in the Trinomial tree framework.
- ▶ Provided potential solutions to the completeness/replicability issue in Trinomial models.
- ▶ Performed pricing of options in the Trinomial tree under assumptions of completeness.

⇒ **TO-DO NEXT:** Go to the next Lesson and see its associated **Jupyter Notebook** to go over a complete pricing example (like the one you have seen conceptually in the slides).

⇒ In the next lessons, we will dig deeper into all these concepts. Also, you will see more realistic coding examples with object-oriented programming that comes in very handy for more complex code down the road.

⇒ Also, please check the extra material provided if you want to see the step-by-step development of the Trinomial tree formulas.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.7
- **Estimated reading time:** 25 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 3

PRICING EXAMPLE IN THE TRINOMIAL MODEL

Derivative Pricing - Module 3



Outline

- ▶ Walkthrough of a full pricing example in the trinomial model with Python.

⇒ **Please go to the Jupyter Notebook.**

Wrap-up of Lesson 3

In Lesson 3, we have:

- Understood pricing in the trinomial tree framework using Python.

⇒ **TO DO NEXT:** Play around with the provided code and try pricing a couple different options.

⇒ In the next lesson, we will develop a more efficient code for this task, making use of object-oriented programming.

Lesson 3 - Pricing Example in the Trinomial Tree

February 8, 2026

0.1 DERIVATIVE PRICING

MODULE 3 | LESSON 3

1 PRICING EXAMPLE IN THE TRINOMIAL TREE

Reading Time	60 minutes
Prior Knowledge	Trinomial model pricing, Matching volatility
Keywords	Trinomial model, Unique EMM, Completeness, Matching volatility

Now that you know the basic intuition regarding pricing in the trinomial tree and the conditions that have to be met (or even imposed) on our end, let's get our hands dirty with a complete example on the trinomial tree.

The code for this lesson will guide you on the process of pricing, from scratch, a European call option under a trinomial tree.

As always, we start importing the necessary libraries:

```
[1]: import numpy as np
```

In order to give a proper (complete) overview of the pricing process, we will start by considering the **match** between the model parameters (u , m , and d) and the **volatility of the underlying asset**.

The following chunk of code, in essence, revisits the matching volatility concept seen in Module 1. Make sure you understand this properly before proceeding. If you need it, feel free to go back to the notebook from Lesson 1.

1.1 1. Matching Volatility in the Trinomial Tree

```
[2]: def _gen_stock_vec(nb, h):  
    s0 = 100  
    sigma = 0.5  
    up = np.exp(sigma * np.sqrt(2 * h))  
    down = 1 / up # down movement to force a "recombining tree"
```

```

vec_u = up * np.ones(nb)
np.cumprod(vec_u, out=vec_u)  # Computing u, u^2, u^3...u^nb

vec_d = down * np.ones(nb)
np.cumprod(vec_d, out=vec_d)  # Computing d, d^2, d^3...d^nb

res = np.concatenate(
    (vec_d[::-1], [1.0], vec_u)
)  # putting together the last period tree underlyings
res *= s0
return res

```

1.2 2. Pricing Derivatives in the Trinomial Tree

Once we have properly understood the matching volatility example, we can proceed with our pricing. To check a baseline case, let's price a **European call option** with the following characteristics:

- $S_0 = 100$
- $K = 90$
- $\sigma = 30\%$
- $r = 0\%$
- $T = 1$
- $N = 2$

Here, you have a picture with the evolution of the underlying asset and the call option payoff at each point of the tree. Remember that we proceeded exactly in the same fashion as with the binomial tree:

- 1- Compute evolution of the underlying.
- 2- Payoff of the call option at maturity.
- 3- Calculate risk-neutral probabilities.
- 4- Expected value (discounted) of call option payoff at each node (backwards induction).

All rights reserved WQU WorldQuant University QQQQ

As you can see, the novel things in this notebook are more related to the practical implementation of the trinomial model in Python than the purely conceptual part of the pricing process.

Let's now look at the actual code used for this pricing:

```

[3]: def price(
    nb_steps,
):  # For now, the only input to the function is the number of steps for the
    ↪tree, N.
    # Define parameters
    r = 0
    sigma = 0.5
    T = 1

```

```

K = 110
h = T / nb_steps # This would be our 'dt' from previous examples
discount = np.exp(-r * h) # Define discount factor for simplicity later on

# Define risk-neutral probabilities:
pu = (
    (np.exp(r * h / 2) - np.exp(-sigma * np.sqrt(h / 2)))
    / (np.exp(sigma * np.sqrt(h / 2)) - np.exp(-sigma * np.sqrt(h / 2)))
) ** 2
pd = (
    (-np.exp(r * h / 2) + np.exp(sigma * np.sqrt(h / 2)))
    / (np.exp(sigma * np.sqrt(h / 2)) - np.exp(-sigma * np.sqrt(h / 2)))
) ** 2
pm = 1 - pu - pd
print(pu, pd, pm)
# This would be our underlying evolution (Note we are using the function
↪ from before!)
s = _gen_stock_vec(nb_steps, h)

# Define Payoff (in this case, European Call Option)
final_payoff = np.maximum(s - K, 0)
nxt_vec_prices = final_payoff

# Proceed with iterations for the calculation of payoffs
for i in range(1, nb_steps + 1):
    vec_stock = _gen_stock_vec(nb_steps - i, h)
    expectation = np.zeros(vec_stock.size)

    for j in range(expectation.size):
        tmp = nxt_vec_prices[j] * pd
        tmp += nxt_vec_prices[j + 1] * pm
        tmp += nxt_vec_prices[j + 2] * pu

        expectation[j] = tmp
    # Discount option payoff!
    nxt_vec_prices = discount * expectation

return nxt_vec_prices[
    0
] # Notice here we only 'return' the expected discounted value of the
↪ option at t=0, that is, the price of the call option!

```

Now, let's check how this code works for different steps in the tree. For starters, let's replicate the solution you have in the picture above, with $N = 2$:

```
[4]: print(" Price of the Euro Call Option is ", price(100))
```

```
0.24124019455888882 0.25891602289479126 0.49984378254631995
```

Price of the Euro Call Option is 16.115031285941384

We can also play around with the **convergence of the price with different time steps**:

```
[ ]: N = np.arange(2, 1000, 50)
prices = []
for i in N:
    prices.append(price(i))
    print(" Price of the Euro Call Option is ", price(i))
```

0.19168941637660364 0.3160424181481997 0.49226816547519664
0.19168941637660364 0.3160424181481997 0.49226816547519664
Price of the Euro Call Option is 16.302102312180924
0.2378953765567142 0.26240498386857664 0.49969963957470914
0.2378953765567142 0.26240498386857664 0.49969963957470914
Price of the Euro Call Option is 16.056824212834997
0.24132572062662855 0.25882743436526606 0.49984684500810533
0.24132572062662855 0.25882743436526606 0.49984684500810533
Price of the Euro Call Option is 16.115423674620974
0.24288264057175957 0.257220141393142 0.49989721803509846
0.24288264057175957 0.257220141393142 0.49989721803509846
Price of the Euro Call Option is 16.1081609411658
0.24382000999378414 0.256257333514394 0.4999226564918218
0.24382000999378414 0.256257333514394 0.4999226564918218
Price of the Euro Call Option is 16.09060235975873
0.24446328702641013 0.25559871181621463 0.4999380011573753
0.24446328702641013 0.25559871181621463 0.4999380011573753
Price of the Euro Call Option is 16.09493708617818
0.24493986498008505 0.2551118698615694 0.49994826515834556
0.24493986498008505 0.2551118698615694 0.49994826515834556
Price of the Euro Call Option is 16.101990300862315
0.24531121793985072 0.2547331686376249 0.4999556134225244
0.24531121793985072 0.2547331686376249 0.4999556134225244
Price of the Euro Call Option is 16.102477248262932
0.24561113720101238 0.25442772894396865 0.499961133855019
0.24561113720101238 0.25442772894396865 0.499961133855019
Price of the Euro Call Option is 16.099656640622545
0.24585994114618945 0.2541746258446278 0.4999654330091827
0.24585994114618945 0.2541746258446278 0.4999654330091827
Price of the Euro Call Option is 16.095132284176962
0.24607067894109808 0.25396044526522665 0.4999688757936753
0.24607067894109808 0.25396044526522665 0.4999688757936753
Price of the Euro Call Option is 16.091334332933375
0.24625216562797234 0.253776139463164 0.4999716949088636
0.24625216562797234 0.253776139463164 0.4999716949088636
Price of the Euro Call Option is 16.096024661258856
0.2464105993482869 0.2536153549030148 0.4999740457486983
0.2464105993482869 0.2536153549030148 0.4999740457486983
Price of the Euro Call Option is 16.098439527360572

```

0.24655048243387703 0.25347348152432614 0.49997603604179686
0.24655048243387703 0.25347348152432614 0.49997603604179686
Price of the Euro Call Option is 16.09926650695079
0.24667517488342988 0.25334708229079633 0.49997774282577373
0.24667517488342988 0.25334708229079633 0.49997774282577373
Price of the Euro Call Option is 16.09896953074233
0.24678724192714308 0.2532335354227728 0.4999792226500841
0.24678724192714308 0.2532335354227728 0.4999792226500841
Price of the Euro Call Option is 16.097869356041826
0.2468886804971464 0.253130801540413 0.4999805179624406
0.2468886804971464 0.253130801540413 0.4999805179624406
Price of the Euro Call Option is 16.096192311818143
0.2469810715041208 0.25303726724933134 0.4999816612465479
0.2469810715041208 0.25303726724933134 0.4999816612465479
Price of the Euro Call Option is 16.09410084601982
0.24706568503635554 0.25295163717996283 0.49998267778368166
0.24706568503635554 0.25295163717996283 0.49998267778368166
Price of the Euro Call Option is 16.093840128545267
0.24714355476935912 0.2528728576865996 0.49998358754404126

```

```

[ ]: import matplotlib.pyplot as plt

plt.plot(N, prices)
#plt.ylim([10.5, 12])
plt.title("Euro Call under Trinomial for diff. N")
plt.xlabel("Number of Steps")
plt.ylabel("European Call Option Price")
plt.grid(True)
plt.show()

```

1.3 3. Conclusion

Now you know how to price a simple option under the trinomial model.

In the next lesson, we will implement an efficient way (using object-oriented programming) to code all these functions.

In the meantime, **we suggest you do two tasks extending this code**. Can you modify this code to price for any N the following options?

- 1 - European Put Option with the same characteristics as before $K = 110$.
- 2 - American Call Option with the same characteristics as before ($K = 90$).

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. freeCodeCamp.org. "Object Oriented Programming with Python - Full Course for Beginners." *YouTube*, 13 Oct. 2021, https://www.youtube.com/watch?v=Ej_02IC0lgs.

- **Required pages:** Entire video
- **Estimated reading time:** 2 hours

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

Lesson 4 - Object-Oriented Programming in the Trinomial Tree

February 8, 2026

0.1 DERIVATIVE PRICING

MODULE 3 | LESSON 4

1 OBJECT-ORIENTED PROGRAMMING IN THE TRINOMIAL TREE

Reading Time	90 minutes
Prior Knowledge	Binomial and Trinomial model pricing dynamics, Python basics
Keywords	Object-oriented programming

—All rights reserved WQU WorldQuant University QQQQ

By now, you know how to price options under a more complex setting such as the trinomial tree. Lesson 4, however, is not as focused on the conceptual part of derivative pricing. Instead, we take advantage of the medium-level complexity of the trinomial model in order to introduce the notion of object-oriented programming (OOP) in the context of derivative pricing. This is purely a programming skill but one you need to master in order to succeed in highly quantitative jobs.

Before going into more complex frameworks, let's take the opportunity now to revisit the previous pricing process under the trinomial model to build our own `TrinomialModel` class algorithm.

Note: In case you are not familiar with OOP (object-oriented programming), [you may want to check out this tutorial on YouTube](#).

The YouTube video tutorial is very long, as it thoroughly goes over the main features of OOP. In spite of its length, it can help to clarify any questions you may have about object-oriented programming.

1.1 1. Let's Start Coding!

As always, start by importing the necessary libraries:

```
[1]: import numpy as np
```

```
[2]:
```

```

class TrinomialModel(object): # Here we start defining our 'class' -->
    ↪Trinomial Model!

    # First, a method to initialize our `TrinomialModel` algorithm!
    def __init__(self, S0, r, sigma, mat):
        self.__s0 = S0
        self.__r = r
        self.__sigma = sigma
        self.__T = mat

    # Second, we build a method (function) to compute the risk-neutral
    ↪probabilities!
    def __compute_probs(self):
        self.__pu = (
            (
                np.exp(self.__r * self.__h / 2)
                - np.exp(-self.__sigma * np.sqrt(self.__h / 2))
            )
            / (
                np.exp(self.__sigma * np.sqrt(self.__h / 2))
                - np.exp(-self.__sigma * np.sqrt(self.__h / 2))
            )
        ) ** 2
        self.__pd = (
            (
                -np.exp(self.__r * self.__h / 2)
                + np.exp(self.__sigma * np.sqrt(self.__h / 2))
            )
            / (
                np.exp(self.__sigma * np.sqrt(self.__h / 2))
                - np.exp(-self.__sigma * np.sqrt(self.__h / 2))
            )
        ) ** 2
        self.__pm = 1 - self.__pu - self.__pd

        assert 0 <= self.__pu <= 1.0, "p_u should lie in [0, 1] given %s" %
        ↪self.__pu
        assert 0 <= self.__pd <= 1.0, "p_d should lie in [0, 1] given %s" %
        ↪self.__pd
        assert 0 <= self.__pm <= 1.0, "p_m should lie in [0, 1] given %s" %
        ↪self.__pm

    # Third, this method checks whether the given parameters are alright and
    ↪that we have a 'recombining tree'!
    def __check_up_value(self, up):
        if up is None:
            up = np.exp(self.__sigma * np.sqrt(2 * self.__h))

```

```

    assert up > 0.0, "up should be non negative"

    down = 1 / up

    assert down < up, "up <= 1. / up = down"

    self.__up = up
    self.__down = down

# Four, we use this method to compute underlying stock price path
def __gen_stock_vec(self, nb):
    vec_u = self.__up * np.ones(nb)
    np.cumprod(vec_u, out=vec_u)

    vec_d = self.__down * np.ones(nb)
    np.cumprod(vec_d, out=vec_d)

    res = np.concatenate((vec_d[:-1], [1.0], vec_u))
    res *= self.__s0

    return res

# Fifth, we declare a Payoff method to be completed afterwards depending on
the instrument we are pricing!
def payoff(self, stock_vec):
    raise NotImplementedError()

# Sixth, compute current prices!
def compute_current_price(self, crt_vec_stock, nxt_vec_prices):
    expectation = np.zeros(crt_vec_stock.size)
    for i in range(expectation.size):
        tmp = nxt_vec_prices[i] * self.__pd
        tmp += nxt_vec_prices[i + 1] * self.__pm
        tmp += nxt_vec_prices[i + 2] * self.__pu

        expectation[i] = tmp

    return self.__discount * expectation

# Seventh, Option pricing!
def price(self, nb_steps, up=None):
    assert nb_steps > 0, "nb_steps should be > 0"

    nb_steps = int(nb_steps)

    self.__h = self.__T / nb_steps

```

```

        self.__check_up_value(up)
        self.__compute_probs()

        self.__discount = np.exp(-self.__r * self.__h)

        final_vec_stock = self.__gen_stock_vec(nb_steps)
        final_payoff = self.payoff(final_vec_stock)
        nxt_vec_prices = final_payoff

        for i in range(1, nb_steps + 1):
            vec_stock = self.__gen_stock_vec(nb_steps - i)
            nxt_vec_prices = self.compute_current_price(vec_stock,
↪nxt_vec_prices)

        return nxt_vec_prices[0]

```

Next, let's define the initialization and payoff methods for a new class (`TrinomialCall`) within the `TrinomialModel` we have just seen:

```

[3]: class TrinomialCall(TrinomialModel):
        def __init__(self, S0, r, sigma, mat, K):
            super(TrinomialCall, self).__init__(S0, r, sigma, mat)
            self.__K = K

        def payoff(self, s):
            return np.maximum(s - self.__K, 0.0)

```

We have defined two classes: `TrinomialModel` and `TrinomialCall`. These allow us to quickly initialize a trinomial model pricing algorithm for the payoff of a European call option. Please take your time in going over the previous chunk of code and make sure you understand it thoroughly.

Now, let's check how our algorithm performs on a simple example with:

- $S_0 = 100$
- $r = 0\%$
- $\sigma = 30\%$
- $T = 1$
- $K = 90$

```

[4]: tree = TrinomialCall(100.0, 0.0, 0.3, 1.0, 90.0)

```

Here, the `tree` variable contains all the different info from the algorithm in `TrinomialCall` class.

How can we see what the price is for, let's say, $N = 2$?

```

[5]: print(tree.price(2))

```

```

17.50162310051333

```

As you see, we got the same number for the price of the Call option as we got from our previous example.

Note that one great thing about object orientation is that now we can define main sub-classes within the `TrinomialModel` as we want, each with a different payoff (e.g., European put option, American, etc.)

1.2 2. Can You Do the Same for a New Class that is `TrinomialPut`?

1.3 3. Conclusion

You now know how to implement all the previous pricing techniques in a more formal way regarding programming. In the next module, we will introduce more formal mathematical notation in order to explore the inherent flaws of simple pricing models based on trees. This will be the first step toward building a more complex model that recognizes the real features of assets.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.